Push Down Automata (PDA)

- A PDA is a way to implement a CFG in a similar way we design a DFA for a Regular Grammar.
- A DFA can remember a finite amount of info, but a PDA can remember an infinite amount of information
- A PDA has three components: (1) an input tape, (2) a control unit, (3) a stack with infinite size.
- A PDA can be formally described as a 7 tuple:

  Q: FNES of states

  Σ: FNES of input symbols ~~($\$ \in \Sigma$)~~

  δ: FNES of transitions

  q_0: initial state

  F: FNES of final states

  S/Γ: ~~sto~~ FNES of stack symbols ~~($\Gamma - Z_0 \subseteq \Sigma$)~~

  Z_0: bottom of stack ~~($Z_0 \in \Gamma$)~~

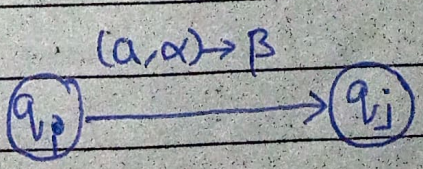
Transition function for a PDA: It takes three inputs — current state, input symbol, popped symbol from stack, and ~~ret~~ gives two outputs — next state, symbol to be pushed in the stack. A PDA transition is of the following format.

$$\delta(q_i, a, \alpha) \longrightarrow (q_j, \beta^*)$$

where, $q_i, q_j \in Q$

$a \in \Sigma$

$\alpha, \beta \in \Gamma$



Instantaneous configuration/description/status of a PDA

The instaneous description of a PDA is represented by the triplet $\{q, w, s\}$ where

q is the current state

w is the unconsumed input

s is the content of the stack

Acceptability in PDA: There are two ways to define acceptability in PDA —

- Final state acceptability
- Empty stack acceptability

i.e.,

A PDA accepts a string, if after reading the entire string, the PDA is in a final state irrespective of the stack content.

OR,

A PDA accepts a string, if after reading the entire string the PDA has emptied its stack.

Construct PDA for the following

- $L = \{a^n b^n \mid n \geq 0\}$

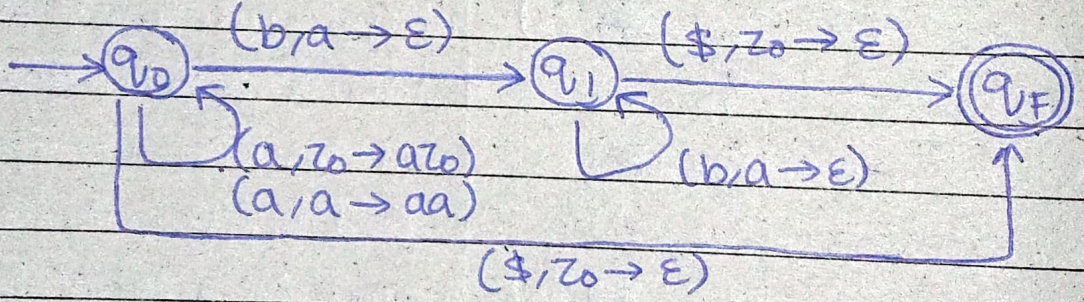
$\delta(q_0, \$, Z_0) \rightarrow (q_F, \varepsilon)$	# desired transitions that are supposed to appear if the input string belongs the given language.
$\delta(q_0, a, Z_0) \rightarrow (q_0, aZ_0)$	
$\delta(q_0, a, a) \rightarrow (q_0, aa)$	
$\delta(q_0, b, a) \rightarrow (q_1, \varepsilon)$	
$\delta(q_1, b, a) \rightarrow (q_1, \varepsilon)$	
$\delta(q_1, \$, Z_0) \rightarrow (q_F, \varepsilon)$	
$\delta(q_0, b, Z_0) \rightarrow (q_D, Z_0)$	# undesired transitions leading us to the dead state. These are supposed to appear if the input string doesn't belong to the given language.
$\delta(q_0, \$, a) \rightarrow (q_D, \varepsilon)$	
$\delta(q_1, a, a) \rightarrow (q_D, \varepsilon)$	
$\delta(q_1, \$, a) \rightarrow (q_D, \varepsilon)$	
$\delta(q_1, a, Z_0) \rightarrow (q_D, Z_0)$	
$\delta(q_1, b, Z_0) \rightarrow (q_D, Z_0)$	

For simplicity one may write only the desired transitions and assume that if a scenario arise that doesn't match with any of the desired transitions, it leads us to the dead state and hence the string is not accepted.

i.e., we can also define a PDA for $a^n b^n \mid n \geq 0$ as follows:

$\delta(q_0, \$, Z_0) \rightarrow (q_F, \varepsilon)$
$\delta(q_0, a, Z_0) \rightarrow (q_0, aZ_0)$
$\delta(q_0, a, a) \rightarrow (q_0, aa)$
$\delta(q_0, b, a) \rightarrow (q_1, \varepsilon)$
$\delta(q_1, b, a) \rightarrow (q_1, \varepsilon)$
$\delta(q_1, \$, Z_0) \rightarrow (q_F, \varepsilon)$

and we diagramatically we can draw it as follows.



- count (a) = count (b)

$\delta(q_0, a, Z_0) \longrightarrow (q_0, aZ_0)$
$\delta(q_0, b, Z_0) \longrightarrow (q_0, bZ_0)$
$\delta(q_0, a, a) \longrightarrow (q_0, aa)$
$\delta(q_0, b, b) \longrightarrow (q_0, bb)$
$\delta(q_0, a, b) \longrightarrow (q_0, \varepsilon)$
$\delta(q_0, b, a) \longrightarrow (q_0, \varepsilon)$
$\delta(q_0, \$, Z_0) \longrightarrow (q_F, \varepsilon)$

- $L = \{a^n b^{2n} \mid n \geq 0\}$

$\delta(q_0, \$, z_0) \rightarrow (q_F, \varepsilon)$

$\delta(q_0, a, z_0) \rightarrow (q_0, aaz_0)$

$\delta(q_0, a, a) \rightarrow (q_0, aaa)$

$\delta(q_0, b, a) \rightarrow (q_1, \varepsilon)$

$\delta(q_1, b, a) \rightarrow (q_1, \varepsilon)$

$\delta(q_1, \$, z_0) \rightarrow (q_F, \varepsilon)$

- $L = \{a^n b^{n+2} \mid n \geq 0\}$

$\delta(q_0, b, z_0) \rightarrow (q_0, bz_0)$

$\delta(q_0, b, b) \rightarrow (q_F, \varepsilon)$

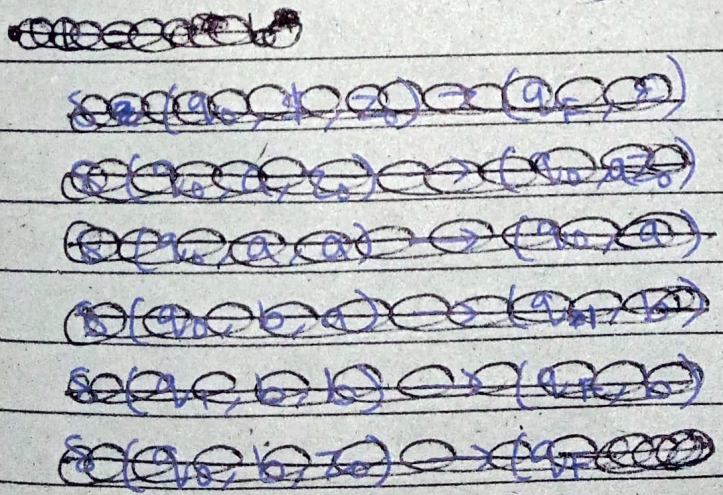
$\delta(q_0, a, z_0) \rightarrow (q_0, aaaz_0)$

$\delta(q_0, a, a) \rightarrow (q_0, aa)$

$\delta(q_0, b, a) \rightarrow (q_1, \varepsilon)$

$\delta(q_1, b, a) \rightarrow (q_1, \varepsilon)$

$\delta(q_1, \$, z_0) \rightarrow (q_F, \varepsilon)$



$L = \{a^n b^m c^n \mid m,n \geq 0\}$

$\delta(q_0, \$, z_0) \rightarrow (q_F, z_0)$ // $m=0$ & $n=0$

$\delta(q_0, b, z_0) \rightarrow (q_F, z_0)$ } // $n=0$

$\delta(q_F, b, z_0) \rightarrow (q_F, z_0)$ }

$\delta(q_0, a, z_0) \rightarrow (q_0, az_0)$

$\delta(q_0, a, a) \rightarrow (q_0, aa)$

$\delta(q_0, b, a) \rightarrow (q_1, a)$

$\delta(q_1, b, a) \rightarrow (q_1, a)$

$\delta(q_1, c, a) \rightarrow (q_2, \varepsilon)$

$\delta(q_2, c, a) \rightarrow (q_2, \varepsilon)$

$\delta(q_2, \$, z_0) \rightarrow (q_G, z_0)$

q_F & q_G are the final states

Conversion of CFG to PDA

- We know that a PDA transition is of the following format:

δ (currentState, inputSymbol, topOfStackPopped)
$\longrightarrow$ (newState, elementToBePushedAsNewTopOfStack)

Step 1: for each production of the given CFG of the type $V_i \longrightarrow \alpha$, add a transition to the PDA of the type

$\delta(q, \varepsilon, V_i) \longrightarrow (q, \alpha)$

Step 2: for each terminal $t \in \Sigma$ of the given grammar add a transition to the PDA of the type

$\delta(q, t, t) \longrightarrow (q, \varepsilon)$

(* step3 on next pg)

eg: $G = \{ A \rightarrow aA \mid bB \mid \varepsilon$
$B \rightarrow bB \mid \varepsilon \}$

Step 1:
$\delta(q, \varepsilon, A) \longrightarrow (q, aA)$
$\delta(q, \varepsilon, A) \longrightarrow (q, bB)$
$\delta(q, \varepsilon, A) \longrightarrow (q, \varepsilon)$
$\delta(q, \varepsilon, B) \longrightarrow (q, bB)$
$\delta(q, \varepsilon, B) \longrightarrow (q, \varepsilon)$

Step 2:
$\delta(q, a, a) \longrightarrow (q, \varepsilon)$
$\delta(q, b, b) \longrightarrow (q, \varepsilon)$

Corresponding PDA

$n(\delta) = n(P) + n(\Sigma)$

Page No. 56
Date

Verification: Let ω = "aaabb" $\in L$

State	Input	Stack	Remark
q_1	aaabb	aA	R_1
q_1	aaabb	aaA	R_1
q_1	aaabb	aaaA	R_1
q_1	aaabb	aaabB	R_2
q_1	aaabb	aaabbB	R_4
q_1	aaabb	aaabb(ε)	R_5
q_1	aabb	aabb	R_6
q_1	abb	abb	R_6
q_1	bb	bb	R_6
q_1	b	b	R_7
q_1	$	Z_0	R_7

Step 3: add the final ~~production rule~~ transition i.e.,

$\delta(q_1, \$, Z_0) \rightarrow (q_1, \varepsilon)$

// this will make the valid strings accepted by the empty stack acceptability criteria since we are not using any final state.